Group 05: Schema-Aware Output-Length Ranking with an Abstaining Reliability Gate for Agentic LLM Serving

Dazhi Yang, Mingye Lang, Yibo Zhang, and Anitha Saravana Kumar
Fairleigh Dickinson University

***Abstract*—Length-aware LLM schedulers read the prompt, or a few statistics drawn from it. Agent traffic carries something they do not read: the JSON tool schemas a tool-calling client attaches to every request, 18–66% of request bytes across the tool sets we captured from a production coding agent, naming tools absent from the ranker's training corpus. We fine-tune an output-length ranker on that schema text and evaluate it in strata defined by how novel each request's tool set is. Reading the schema is worth $+0.044$ Kendall's τ over the same encoder given the prompt alone (95% paired CI $[+0.024, +0.066]$); the text encoder as a whole is worth $+0.19$ over the tuned scalar features it replaces, and quality stays at τ=0.64–0.65 where the tool set is partly or wholly unseen (equivalence untested), which is where identity-based features have nothing to look up.**

A better order becomes lower latency only where the engine leaves a queue to reorder, and across our runs nothing but an engine setting decided whether it did. With vLLM's running-batch cap at its default of 256 slots the waiting queue was empty at the 90th percentile, and no ordering policy separated from another by more than 1.2%. Capping the batch at sixteen puts 57–64% of requests behind another, and the same policies then separate: the ranker beats arrival order by 4.2% (95% CI $[0.944, 0.972]$), while ordering by prompt length, which costs nothing to compute, runs 8.5% slower than not reordering at all. The gate does not pay either — abstained requests hold their slots and block those behind them, leaving the gated system 4.9% slower than arrival order itself. Nor is the signal free: under the gateway's documented 15 ms decision timeout a third of the scores arrive too late to use, though a budget recalibrated to the measured scorer loses none by 75 ms. Neither ordering nor prediction produced the largest effects we measured, and both are capabilities the engine already ships: moving from its synchronous scheduler to the asynchronous base our policies inherit cut mean time-to-last-token by 44% with arrival order held fixed, and prefix caching cut a further 17–23%.

I. Introduction

When a coding agent talks to a language model, much of what travels over the wire is not the user's question. Intercepting a production coding agent (OpenCode) with an echo endpoint, we measured a single agent-loop request. It declared 170 tools whose JSON schemas occupied 147 kB, 67% of that payload, before the model read a word of the task. The share is a range, not a constant. Within that agent's own tool set it runs 17–57% of request bytes (median 23.5%), falling as conversation accumulates around a schema block that is resent whole each turn; across every tool set we captured the range widens to 18–66%. Figures use canonical serialization; raw wire bytes differ by roughly two points. No length predictor in the line we survey reads any of it.

Reading it would matter because schedulers want to know how long each request will run. Shortest-job-first ordering needs an output-length estimate, and a line of work has learned such estimates from prompt text alone [1], [2]. The traffic we captured violates the assumptions behind both the scalar and the prompt-only designs. A third of its requests carry no tools (title generation and other utility calls), each client's schema is byte-constant across turns of the same call kind, and the tool vocabulary in the wild is open-ended, so a learned component must handle tools it has never seen. In our held-out evaluation, 95.5% of test requests carried a tool combination never seen in training, and every tool-bearing request captured from the live agent was composed entirely of tools foreign to the training corpus.

Our deployment context is an API gateway. Gateways are an established tier in classical service infrastructure, where one layer owns load balancing, admission control, caching and failure isolation, and operators now place that same tier in front of LLM traffic with little modification. Agent requests in this study reach the model through such a gateway (VeloxMesh), which owns routing, admission and provider health, and which consults our scoring service at admission time (Fig. 1). The gateway treats scoring as optional by design: a documented 15 ms budget bounds the decision, and on timeout or error the request proceeds in arrival order. The scheduling work we survey optimizes inside the engine and does not address this budget, so a learned predictor must fit inside it, recalibrate it, or move off the synchronous path. This paper asks which of the gateway's classical mechanisms remain useful for agentic LLM traffic, and whether a learned signal can operate within the budget the gateway grants it.

We argue for two design changes. The request should be read as *text*: a ranker that consumes prompt and schema text orders unseen-tool requests with little observed τ difference across the two adequately powered unseen-tool strata (equivalence untested), while the identity features we tested have nothing to look up there at all and the tuned scalar family ranks unseen tools only from well behind it. Prediction should also carry its own error bounds: our reliability gate assigns each request a confidence measured per cold-start stratum and abstains, preserving arrival order, wherever that measurement does not exist.

This paper contributes:

- a workload characterization of live agent traffic: across

captured tool sets, 18–66% of request bytes are unread tool schema (67% in the agent's 170-tool configuration), about 2 kB per tool in its native sets, resent every turn; 33% zero-tool requests; schemas byte-constant within a call kind; and a tool set entirely out of vocabulary relative to the corpus we train on;
- schema-text ranking: $+0.19\ \tau$ over a grid-searched scalar baseline — a figure that changes both the model and its input, of which $+0.044$ is the schema text alone against the same encoder reading the prompt only — with no drop detected across the two adequately powered unseen-tool strata (equivalence untested), while schema-identity baselines gain $+0.008$ (within noise) and *lose* accuracy when forced to use the identity feature;
- a decision-budget coverage curve: neither un-truncating the schema nor caching its encodings makes the CPU scorer fit the documented 15 ms default, which it misses on essentially every request; even served from the GPU, 34.6% of traffic still fails open silently at that budget; a timeout matched to the measured GPU scorer (50–75 ms) restores 98–100% coverage, permitting that much decision latency on the critical path of a scored request rather than incurring it;
- a contention threshold for learned ordering: with the engine's running-batch cap at its default the queue never forms and no policy separates, but capping it at sixteen puts 57–64% of requests behind another, and there the learned ranker beats arrival order by 4.2% while ordering by prompt length runs 8.5% slower than not reordering at all;
- a serving-level attribution: switching the scheduler implementation alone moved mean latency 44% with arrival order held fixed, and prefix caching a further 17–23%, both measured at the default batch cap;
- an evidence-based abstaining gate: per-stratum confidences derived from measured ranking quality replace a hardcoded constant that overstates reliability by $+0.25$–$+0.46$ everywhere; the lowest observed ranking quality occurred on *new combinations of familiar tools* (n=78, below our reporting threshold).

We evaluate on tool-calling corpora and on a live gateway stack serving Qwen3.5-9B, with pre-registered criteria fixed before each measurement. At serving time our pre-registered superiority test fails: no ordering policy separates from another by more than 1.2%, while swapping the scheduler implementation alone cuts mean latency by 44%. We report this attribution result in place of the improvement we expected.

The remainder of this paper is organized as follows. Section II provides serving background and defines our vocabulary; Section III positions this work against prediction-based, prediction-free, and imprecise-information scheduling; Section IV details data, models, and pre-registered criteria; Section V presents workload characterization, ranking, cold-start, cost, and gate results; Section VI discusses design consequences and limitations; Section VII concludes.

II. Background

A. Serving, Scheduling, and Head-of-Line Blocking

Modern LLM engines admit requests continuously and batch them at token granularity; continuous batching [3] and vLLM's paged KV-cache [4] are the substrate for this work. The scheduler decides which waiting request joins the running batch next. First-come-first-served is the default, and its weakness is classical: a long-running request admitted early delays every short request behind it. A 2026 survey of latency in LLM agent systems independently lists FIFO-induced head-of-line blocking among live system bottlenecks for agentic traffic [5], while placing learned scheduling in its future-work section — the gap this paper works in.

B. The Running Batch and the Waiting Queue

An engine that batches continuously holds two sets of requests, and only one of them is schedulable. The *running batch* is what the GPU steps forward on the current iteration; the *waiting queue* holds requests that have been admitted but hold no slot yet. vLLM caps the running batch at `max_num_seqs`, 256 by default. A scheduler chooses which waiting request enters the batch as slots free, so it can only express an ordering over requests that are already waiting. When the cap is never reached, every arrival enters the batch at once and the queue a policy would reorder never forms.

Whether it forms follows from Little's law: the mean number in flight is the arrival rate times the mean time each request spends in the system. Serving 3.96 requests per second at a mean of 2.75 s holds about eleven at a time, and 6.4 per second holds about nineteen in our runs — neither near 256, and no rate we could sustain on one card would be. The cap therefore decides whether the scheduling problem this paper studies is present at all, which is why Section V reports two regimes.

C. Learned Output-Length Prediction

Shortest-job-style ordering requires knowing output lengths in advance. Since generation length is stochastic, a body of work predicts it: early designs classify responses into coarse length buckets or predict sequence length with a proxy model [6]; the learning-to-rank line [1] showed that *relative order*, not absolute length, is what scheduling needs, training a small proxy with a ranking loss; PARS [2] refined this with pairwise margin training over a BERT encoder. All of these read the prompt (or scalar request statistics); none consume tool-schema text.

D. System Substrate: Gateway and Decision Service

Our serving stack (Fig. 1) fronts the engine with an LLM gateway (VeloxMesh) that owns routing, admission, and a scheduler hook with a documented default timeout: scoring must return within 15 ms (operator-configurable), after which the gateway *fails open* to arrival order. The gateway consults a separate decision service over HTTP; that service hosts the Ranker and applies the Reliability Gate on the gateway's behalf. This fail-open contract is load-bearing for everything

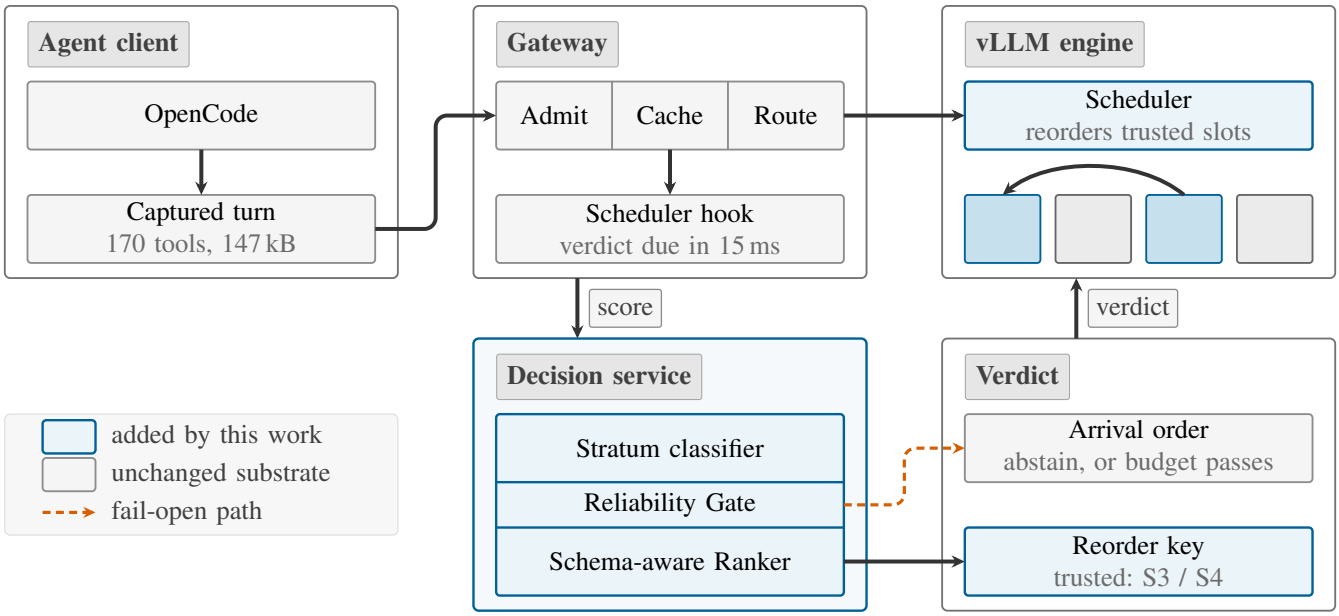
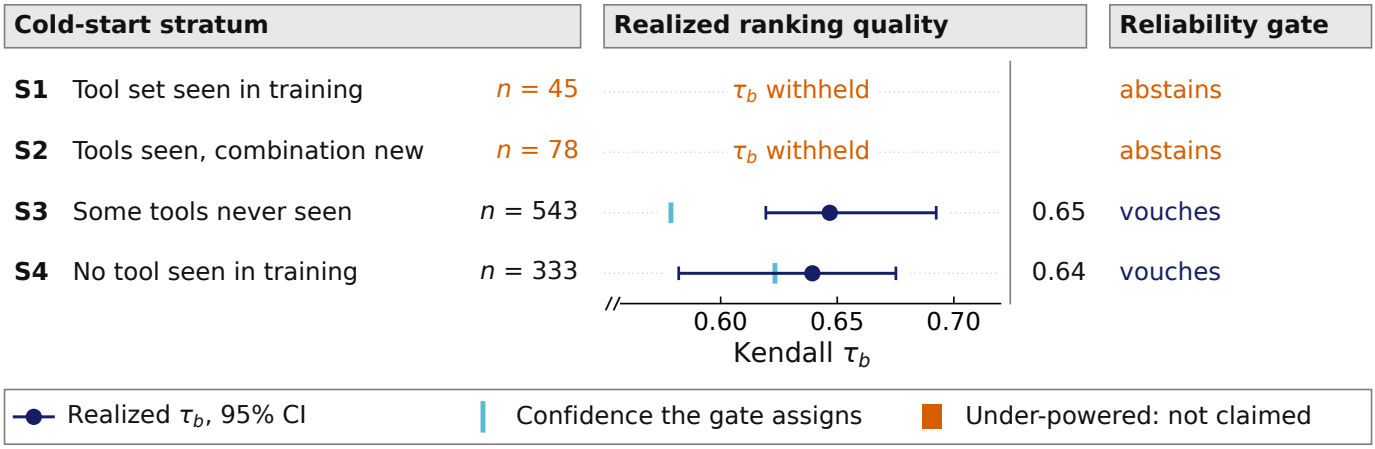


Fig. 1. The gateway scores at admission inside its budget and forwards the verdict; the engine's scheduler reorders trusted slots among themselves and leaves abstained ones where they arrived, and a timeout or an abstention yields arrival order. The 15 ms shown is the gateway's documented default, not a fixed cost: it is operator-configurable, and at 75 ms none of our scores arrive late.



Markers are the mean over 3 seeds; bars are 95% bootstrap CIs (seed-17 scores). Axis truncated; full range [−1, 1].

Fig. 2. The four cold-start strata, and which of them the paper is allowed to speak for. S1 and S2 hold too few test requests to support a headline claim, so their Ranking Tau is not plotted and the gate routes them to Fallback; both values appear with their sizes in Sec. V, where they inform the gate's design rather than a claim.

that follows: an unreliable or slow prediction never blocks the request path.

E. Vocabulary

Throughout, the *Ranker* is the fine-tuned BERT model scoring a request's expected output length; the *Reliability Gate* is the runtime decision of whether that score may influence queue order; *Fallback* means serving in arrival order because the gate withheld trust. *Ranking Tau* is Kendall's τ_b [7] between the predicted and realized output-length order of every held-out pair, running from -1 through 0, which is what guessing achieves, to $+1$; it scores order rather than token error, and 0.64 corresponds to agreeing on roughly four pairs in five. Because it is computed over all held-out pairs while a scheduler chooses only among requests waiting together, it is evidence that a usable signal exists in the input, not that any latency will improve. *Cold-Start Transfer* measures it on test strata whose tool sets were unseen in training, within the same workload family: S1 (seen combination), S2 (new combination of seen tools), S3 (partially new tools), S4 (all-new tools); Fig. 2 gives each stratum's size and what the gate does with it. It is distinct from *Cross-Workload Transfer*, which changes

the workload family entirely.

III. Related Work

A. Learned Length Prediction for Scheduling

A substantial line learns output-length signals to approximate shortest-job-first serving [6], [1], [2], [8], progressing from length-bucket classification to relative-rank training, the insight we inherit: scheduling needs *order*, not absolute length [1]. PARS, now peer-reviewed at ISC High Performance 2026, pairs a BERT encoder with pairwise margin training and demonstrates cross-model robustness [2]. However, every predictor in this line reads the prompt or scalar request statistics; none consumes tool-schema text, and none is evaluated on requests whose tools were absent from training. Our work fills this gap: schema text as predictor input, and a novelty-stratified evaluation in which 95.5% of test requests carry tool combinations unseen in training — the regime our single-client live capture exhibits throughout.

B. Prediction-Free Scheduling

A newer position holds that tail latency does not need prediction at all. UniBoost's distribution-aware boosting beats even an *oracle*-length SRPT baseline by 35.1% on P99 time-to-last-token in its Table 2 configuration [9], and starvation-free preemptive designs make adjacent arguments [10], [11]. We therefore make no tail claims: if an oracle loses on P99, better ranking accuracy will not repair that, so our endpoints are pooled means rather than tail percentiles. The concession is scoped by the papers' own designs, however: UniBoost evaluates no tool-calling workload, disables prefix caching for experimental control, and is prediction-free but not feature-free — its scheduler consumes prompt length, the same class of signal our weakest baseline uses. Whether its result transfers to agentic traffic is, by its own construction, open.

C. Scheduling Under Imprecise Information

JITServe is the strongest published treatment of acting on unreliable estimates: quantile-forest *upper bounds* on length, refined every $\sim$50 generated tokens, drive grouped goodput scheduling within 3–9% of an oracle on a 16-GPU cluster — with BERT-class predictors explicitly rejected at 56–187 ms under load against a forest costing 11–24 ms at those same loads (its oft-quoted 7 ms is the lightest, 8-requests/s point) [12]. Its tool awareness is structural: tools enter as identity and duration in a dependency graph, never as text into the length model. We position our gate as complementary, not competitive. JITServe absorbs uncertainty by always scheduling against a pessimistic bound; our gate *detects* where trust has no measured basis and abstains, preserving arrival order — the "explicit fallback policies" its authors name as an open problem under distribution shift. Agent-serving systems that exploit workflow structure [13], [14] are orthogonal: they schedule around tool-call *interruptions*, not around length uncertainty.

D. Schemas as Model Input; Unseen-Tool Evaluation

Fine-tuning small encoders on schema text is established practice in tool learning — it is how function-calling models are trained [15], [16], [17] — and Switchcraft packs function signatures into a DistilBERT to route requests across models by predicted *correctness* [18]. The distinction we claim is therefore narrow and specific: schema text $\rightarrow$ output-length rank $\rightarrow$ queue order is, to our knowledge, unoccupied; Switchcraft itself lists benchmark-shift quantification as future work; our Cold-Start Transfer strata measure that axis. For the stratified design we follow tool-learning precedent: ToolLLM evaluates at unseen-instruction, unseen-tool, and unseen-category levels [16], licensing our S1–S4 *design* though not our metric (it scores task success, not length rank). BFCL's crowd-sourced split targets contamination control, a different axis [19], and Gorilla's holdout is random — its "zero-shot" denotes retriever absence, not unseen APIs [17] — so neither substitutes for tool-novelty stratification.

E. The Serving Substrate: Gateways, Admission, Prefix Reuse

Our contribution sits between two systems layers that are usually studied separately. Below the engine, continuous batching [3] and paged attention [4] set the execution substrate, and prefix reuse — RadixAttention in SGLang [20] and the prefix caching now standard in vLLM — exploits the repetition our capture exhibits, a schema resent byte-identically on every turn of a given call kind; our default-cap ordering runs hold it disabled to isolate ordering, one round enables it to measure it, and the capped-batch round holds it on in every arm. Above the engine, gateways own admission: overload control in production RPC systems sheds load by priority when queues exceed bounds [21], and the gateway we deploy on implements the same pattern as queue guards with priority bypass. Admission decides *whether and when* a request reaches the engine; the engine's scheduler decides *which* waiting request runs next. Our predictor feeds the second decision, and its budget is set by the first, so a scoring path slower than the configured admission timeout is excluded regardless of its ranking accuracy; the timeout itself is an operator choice.

IV. Methodology

A. Data and Labels

Training and offline evaluation use a 6,000-request stratified sample of ToolACE tool-calling conversations, labeled with completion lengths measured against Qwen3.5-9B under frozen generation controls (temperature 0, top-p 1, seed 42, max 4096 tokens). We chose Qwen3.5-9B because it is open-weight, carries first-class tool-calling support in vLLM (native tool-call and reasoning parsers), and at 9B serves in bf16 on a single 48 GB card with 16k context. Using the same model for labeling and serving keeps the label distribution matched to the engine under test. Three rows whose labeling runs failed are declared as structural exclusions rather than silently dropped, and three censored rows are excluded, leaving 5,994 eligible rows split 3,997/998/999 by a per-row split column

TABLE I. Feature variants and offline recipe. All variants share one fixed 3,997/998/999 split of the labelled ToolACE sample.

Variant	Ranker input	Note
prompt_only	prompt text	encoder-strength control
prompt_schema	prompt + schema text	the proposed input
full_context	+ history	void under truncation
scalar (LightGBM)	5 request statistics	fixed + 20-config grid
identity (hash)	scalar + tool-set fingerprint	lookup emulation

fixed before any experiment; every run in this paper shares that split. Label censoring is 0.05%.

B. Ranker and Feature Variants

The Ranker is BERT-base [22] producing a scalar score $s_\theta(x) \in [0,1]$ from input rendered as `[USER]` prompt `[TOOLS]` schema, truncated at 512 tokens. We fine-tune with the pairwise margin objective

$$\mathcal{L}(x_a, x_b) = \max\big(0,\ -y \cdot (s_\theta(x_a) - s_\theta(x_b)) + m\big), \quad (1)$$

where $y = \pm 1$ encodes which request produced the longer completion, m=1.0, and pairs enter training only when their normalized length gap exceeds δ=0.2; three seeds (17/42/73) vary initialization and pair shuffling over one fixed split. Table I lists the Feature Variants; the full-context arm is reported void because the same 512-token cap leaves 80% of its inputs truncated — a disclosure, not a result.

C. Baselines and Identity Features

The scalar baseline is LightGBM [23] over prompt length (chars, words), schema length, tool count, and history turns — the scalar family gateway-side length predictors describe — in a fixed-hyperparameter form and a 20-configuration grid search whose configuration is chosen on the validation split, never on test; the stronger result is the baseline of record. The recipe is deterministic with respect to seed (no stochastic sampling consumes the random state), verified by bit-exact rerun. Identity baselines fingerprint the tool set as SHA-256 over sorted tool names — never over the raw schema string, which embeds per-request timestamps that would fake uniqueness — and add out-of-fold per-fingerprint label statistics, emulating a deployed per-client lookup table with a global-mean fallback for unseen keys.

D. Cold-Start Strata and the Gate

The fixed test split is partitioned into S1–S4 by fingerprint and tool-name novelty against the training vocabulary. Strata under 100 rows (S1=45, S2=78 at test) are withheld from headline claims; they may inform design when disclosed with their size. The abstention set itself is fixed on the validation split alone: S1 and S2 fall below the same 100-row floor there (n=74 and n=82, against 472 and 370 for S3 and S4), so the gate has no validation basis on which to vouch for them. Their test sizes are reported for power, never used to set the rule. The Reliability Gate assigns each stratum the lower bound of the 95% session-clustered bootstrap [24] confidence interval of *validation* Ranking Tau — fit on validation, evaluated on test — and abstains (confidence 0) on strata it cannot measure, including zero-tool requests:

$$c(x) = \begin{cases} \max\big(0,\ \underline{\tau}_{95}(S(x))\big) & S(x) \in \{\mathrm{S3}, \mathrm{S4}\} \\ 0 & \text{otherwise,} \end{cases} \quad (2)$$

where $S(x)$ is the request's stratum under the training vocabulary and $\underline{\tau}_{95}$ the lower bootstrap bound of validation Ranking Tau. A request is trusted when $c(x) \geq 0.5$; abstained requests keep their arrival slots, and trusted requests reorder only among trusted slots. This confidence is a stratum-level abstention score, not a calibrated probability.

E. Serving Configuration

The gateway runs in its default configuration plus exactly six environment variables: listen address, API key, upstream provider and model, the decision endpoint, and its timeout. Its request-shaping pipeline, provider combos, semantic cache and scheduler-side queue limits are all at their disabled defaults. The decision timeout is 6,000 ms during ordering runs — deliberately far above the documented 15 ms scheduling budget, so that fail-open never fires; differences between ranked and unranked arms then bundle the ranking signal with the synchronous scoring cost that carries it, and are read as such. The budget itself is enforced and measured in dedicated arms at 15 ms (the documented default), 50 and 75 ms (≈1.3× and 2× the Ranker's GPU median), tracing in-budget coverage against the operating point an operator who had measured our scorer would actually set. vLLM prefix caching is disabled uniformly across the default-cap ordering arms, verified from the engine's reported hit rate rather than from the flag alone. It is enabled in the arm that measures its effect and in all four arms of the capped-batch round, where holding it on everywhere means it cannot separate them.

F. Serving Environments and Provenance

Table II pins both environments. Serving experiments run on a single cloud RTX 4090 with 48 GB VRAM (a vendor-modified memory configuration common on Chinese GPU clouds); the decision service runs the Ranker in fp16 with micro-batching (batch $\leq$ 8, 3 ms window) and a reliability threshold of exactly 0.5. All vLLM capacity and cache settings are held constant across every arm of every comparison; prefix caching is disabled in every default-cap ordering arm — verified from the engine's own reported hit rate rather than from the flag alone — and enabled in the round that measures it and throughout the capped-batch round. The scheduler is replaced through vLLM's [4] `--scheduler-cls` seam. Each policy arm subclasses the engine's `AsyncScheduler` and re-sorts only the waiting queue before each step, all else stock code; the stock arm runs the default synchronous scheduler, a difference E5's attribution control measures.

V. Evaluation

A. E1: What Agent Requests Are Made Of

Fig. 3 characterizes the capture. It covers 75 requests across 25 tasks of a production coding agent. Each tool in the native

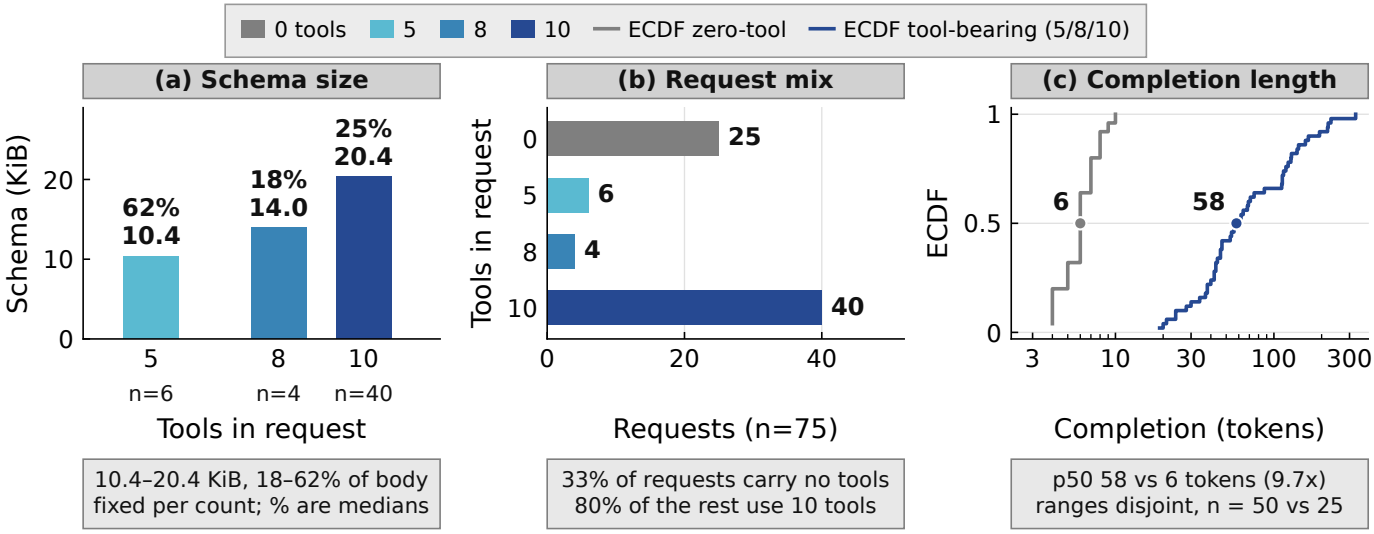


Fig. 3. Captured agent traffic: across the captured tool sets schemas are 18–66% of request bytes, a third of requests carry no tools, and completion medians split 6 versus 58 tokens, so the pooled median of 42 describes neither kind.

TABLE II. Environment provenance for the two measurement platforms, which are separate rentals: labelling and fine-tuning ran first, serving weeks later.

	Training	Serving
GPU	RTX 3090 24 GB	RTX 4090 48 GB (modified)
Driver / CUDA	—	580.159.03 / 13.0 wheels
Engine	vLLM 0.19.1, torch 2.10	vLLM 0.24, torch 2.11
Model	Qwen3.5-9B (labels)	Qwen3.5-9B, bf16
Scheduler seam	—	`-scheduler-cls` per arm

sets contributes about 2 kB of schema (1.8–2.1 kB), giving 21 kB for ten native tools; the denser 170-tool configuration totals 147 kB, resent whole every turn. That fixed per-turn cost is an unstable share of the request: 17% to 57% within the native configuration alone (median 23.5%), and 67.0% with 170 tools. Within a call kind the tool set is byte-identical across turns, and the 75 requests carry only three distinct sets. Against the ToolACE vocabulary, every tool-bearing live request classified as S4.

B. E2: Schema Text vs. Scalars vs. Identity

Fig. 4 gives the headline offline result. Reading the request as text beats the scalar baseline family by $+0.19$ τ against the tuned baseline of record (0.6302 vs. 0.4395; fixed form 0.4268). The prompt-only control reaches 0.5865, so roughly 77% of that gap comes from the text encoder and 23% from the schema text. Isolated, the schema is worth $\Delta\tau_b = +0.0437$ (95% session-clustered paired CI $[+0.024, +0.066]$). Schema identity is nearly worthless: a fingerprint-plus-lookup baseline gains $+0.008$, inside its own ± 0.035 interval, because only 4.5% of test requests share a fingerprint with training.

The same models are evaluated on the two strata Fig. 2 clears for use: S3 (partially new tools, n=543) and S4 (all-new tools, n=333). Both pre-registered criteria pass on both strata against the tuned baseline, and schema-text τ differs by only 0.0075 between them (0.647 and 0.639, against 0.630

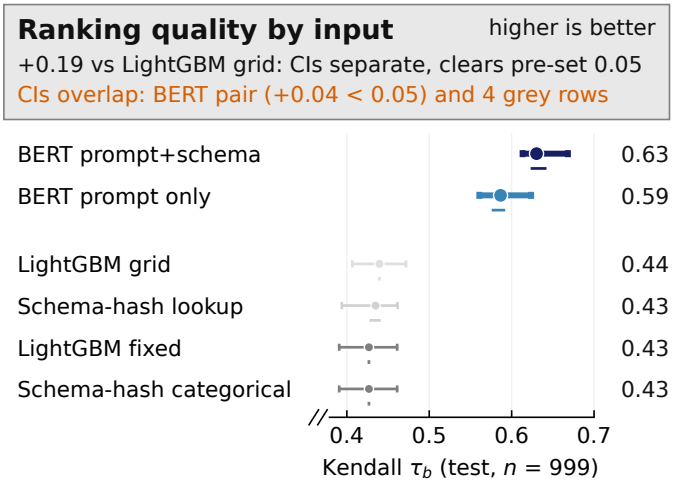
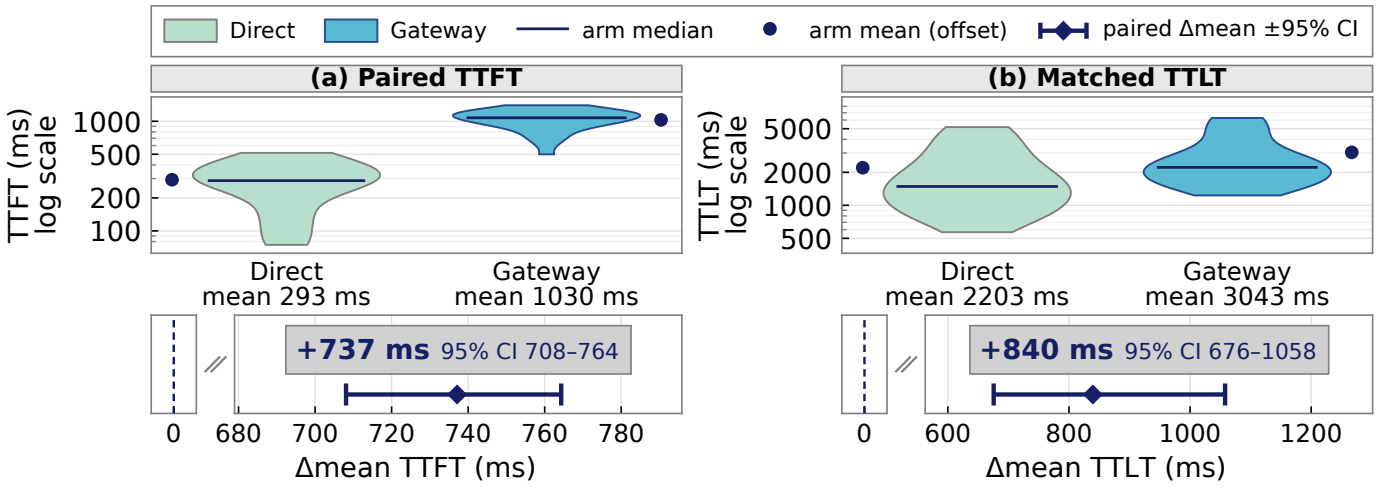


Fig. 4. Ranking quality by input family: prompt-plus-schema text exceeds the tuned scalar baseline by $\Delta\tau_b = +0.191$. Dots are the mean over 3 seeds; capped bars the 95% bootstrap CI on seed 17 alone; the lane below each row the min–max across seeds, a tick where that spread is zero.

overall). We ran no equivalence test, so this is consistent with no degradation rather than a demonstration of it. What does not hold flat is the *margin*: schema text leads the tuned baseline by $+0.25$ on S3 but only $+0.14$ on S4, because the scalar baseline itself scores higher where no tool is familiar (τ_b 0.50 against 0.40). Reading the schema keeps its own quality on wholly unseen tools; it does not widen its lead there.

C. E3: Deployment Cost and Two Failed Rescues

Fig. 5 prices the admission path the scorer sits on. The Ranker costs 722 ms at p99 on production CPU settings at the deployed concurrency of eight, 48× the gateway's 15 ms contract and still 8× over it served serially. Two pre-registered



(b) 32 of 150 pairs dropped for unequal output tokens; direction of the residual bias is unknown.

Fig. 5. Admission-time scoring versus direct: +737 ms TTFT and +840 ms TTLT over the whole added path. Panel (a) is 150 paired requests and (b) the 118 of them with equal output tokens; violin bodies and y-axes are trimmed to p5–p95 of each arm, and intervals are 2,000× bootstrap percentile CIs.

rescues fail. *Un-truncation*: 71% of schemas exceed the 512-token cap, yet a two-tower encoder reading the whole schema moves τ by $+0.0008$ (S3) and -0.0112 (S4), intervals overlapping. *Caching*: precomputing the schema tower yields 1.9×, still 25× over contract at concurrency 8 (4.3× serial). On the serving GPU with fp16 micro-batching, scoring costs 37.9 ms at p50, 6.0× faster than unbatched GPU inference and 16.6× faster than the CPU path at the same percentile (628.8 ms p50). Enforcing a budget end to end is a different measurement, and we report it over every request the gateway handled rather than over the scored subset: under the documented 15 ms default, 801 of 2,315 requests (34.6%) fail open into arrival order, silently; at 50 ms that falls to 2.0% and at 75 ms to zero. The denominator includes requests the gate never scores at all — the zero-tool third of the traffic and the strata it abstains on — so 34.6% is the share of *traffic* left unordered, not the share of scoring attempts that miss the budget, which is necessarily larger.

D. E4: Assigned versus Realized Confidence

Fig. 6 contrasts the confidence each rule assigns with the ranking quality it actually delivered. The placeholder constant confidence (0.9) overstates realized ranking quality on every stratum, by $+0.25$ to $+0.46$; the lower CI bound of validation τ per stratum, abstaining where it cannot be measured, did not overstate on any stratum here: its closest approach to overstating was -0.016 on S4, and its largest understatement -0.068 on S3. S2 is where the Ranker fails: familiar tools in new combinations realize $\tau = 0.44$, against 0.60 on S1, 0.65 on S3 and 0.64 on S4 — 0.16 to 0.21 below every other stratum. At n=78 that sits below our reporting threshold, so we read it as a design signal rather than a result, which is why Fig. 2 plots neither it nor S1's.

E. E5: Serving-Level Scheduling

We replayed 2,075 requests open-loop at 3.96 requests/s, 90% of the highest rate the stock scheduler sustained under a 95% completion criterion (4.4 requests/s). Six schedulers share one arrival schedule: stock first-come-first-served, a shim wrapping it unchanged, and four on our own base, *PolicyFCFS* (arrival order, no scorer), *PromptLengthSJF* (prompt length), *PureLTR* (Ranker score) and *GatedRuleC* (Ranker score where the gate vouches). The pre-registered endpoint is pooled mean time-to-last-token (TTLT), compared as paired-mean ratios with hierarchical bootstrap intervals against a pre-declared 3% margin; time-to-first-token (TTFT) is reported only where the admission path is priced.

The attribution control comes first (Fig. 8). *PolicyFCFS* and stock serve the same requests in the same arrival order and differ only in scheduler implementation; PolicyFCFS records a paired mean-TTLT ratio of 0.560 ($[0.540, 0.581]$), so that 44% reduction arrives with the switch of implementation itself, none of it contributed by ordering. Interposition does not explain it: the shim lands within 2 ms of stock (4903 vs. 4905 ms). What does differ is the base class: the policy arms subclass vLLM's `AsyncScheduler`, which overlaps scheduling with model execution, while stock and the shim run the synchronous one — the likeliest mechanism, though the one-line ablation confirming it did not make our budget. The mode's trade is explicit: the GPU never waits for the scheduler, and in exchange each step is planned on state one step old — completions free their slots a step late, arrivals join a step later, and a token scheduled past a stop is discarded. Timing changes; output never does.

Against the same-base control, ordering does not move the endpoint here. The superiority test fails: GatedRuleC versus PromptLengthSJF gives 1.008 ($[0.979, 1.041]$), an interval containing one. Safety passes: against PolicyFCFS the ratio

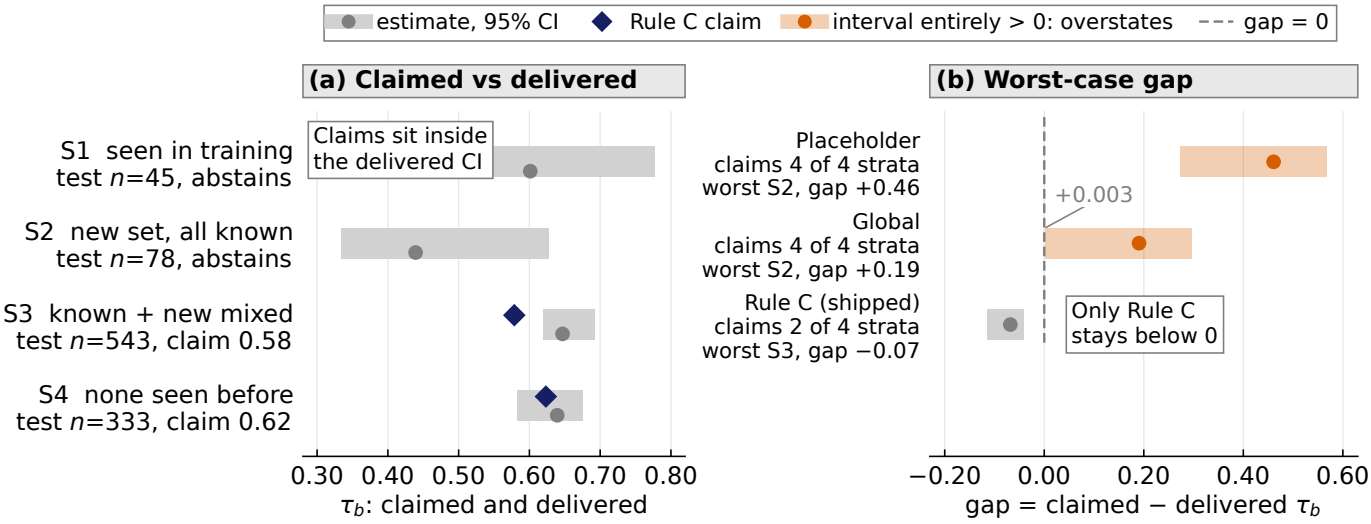


Fig. 6. Assigned versus realized confidence: the placeholder constant overstates on every stratum; Rule C did not on this split, at the cost of abstaining where it cannot measure.

is 0.996 ($[0.970, 1.023]$), inside the 3% margin, and with an empty queue the gate itself costs at most 1.4% (GatedRuleC against PureLTR, 1.003, $[0.991, 1.014]$; Sec. V-F).

The order logs quantify how often ordering could act and show that the policies acted when able. Across 77,000 scheduling steps per arm the waiting queue was empty at p90 and held two waiting requests in under 1% of steps; 11.9–12.5% of requests first entered a queue of depth two or more. Expressing an order needs two requests that were *already* waiting to meet in the same step. That happened in 293–324 steps per arm, and the policies rewrote 262–307 of them (87–95%), while *PolicyFCFS* rewrote none. Ordering was executed on nearly every opportunity it had and mean TTLT did not move. This round did not isolate why: three mechanisms are consistent with it, the replay's compressed job-size variance, a two-deep queue where a swap moves a request by at most one slot, and residual ranking error at τ=0.63. Round A separates none of them; E6 does.

A second round replays all six arms with vLLM prefix caching enabled and nothing else changed. The stock arms bridge the sessions, their on/off ratios agreeing to four decimal places (0.8291 vs. 0.8290), and the engine's log confirms the cache took effect (62.1% peak hit rate). Caching cuts pooled mean TTLT by 17.1% on the stock pair and 22–23% on the four policy-base arms. The policy intervals overlap the bridge's, so the larger gain is suggested rather than established, and the experiment did not isolate the schema's share of it.

A third round at 6.4 requests/s in three blocks of cold engine launches fails the same check (exposure only 14.3–15.0%, *not one* carry-over step across twelve launches), so it is non-diagnostic for superiority. On those independent launches GatedRuleC gives 1.001 ($[0.988, 1.015]$), 1.006 ($[0.993, 1.020]$) and 1.006 ($[0.995, 1.018]$) against PromptLengthSJF, PolicyFCFS and PureLTR, all inside the 3% margin.

F. E6: Ordering Under Contention

None of those rounds formed a queue: the running-batch cap sat at vLLM's default of 256 slots while only eleven to nineteen requests were in flight. E6 caps it at 16 and repeats the question at 6.4 requests/s with prefix caching on and one cold launch per arm. Three conditions besides the cap differ from round A — rate, caching, launch structure — so the panels compare regimes, not otherwise-identical runs; held exactly are the workload, arrival offsets, checkpoint and the four policies, which keeps the ordering comparison *within* each regime clean. The cap is an instrument, not a recommendation. A scheduler can only act where requests wait, and what we could vary at our budget was the number of slots rather than the arrival rate needed to fill 256 of them; a deployment reaches the same condition whenever offered concurrency exceeds the slots it has, whether because the cap is low or because long contexts exhaust the KV cache first. What E6 tests is therefore the regime, not the number: the claim is that ordering pays once a queue exists, and the cap is how we made one. The manipulation check passes this time: 57.5–63.6% of requests entered a queue of depth two or more, p90 depth reached 8 to 14, and 5,914 to 7,121 steps per arm carried two requests that had both waited earlier, against 293–324 in round A. The policies rewrote 1,817 to 6,607 of them, and *PolicyFCFS* again rewrote none.

Ordering now moves the endpoint (Fig. 7), and the pre-registered primary test passes for the first time: GatedRuleC versus PromptLengthSJF gives 0.966 ($[0.935, 0.998]$), an upper bound below one. What pays is the learned signal. PureLTR beats arrival order by 4.2% (0.958, $[0.944, 0.972]$), while the free length heuristic runs 8.5% *worse* than doing nothing (1.085, $[1.051, 1.122]$), so rendered prompt length is a poor proxy for output length; over 2,025 shared sessions pooled mean TTLT is 2.797 s for PureLTR against 3.169 s for PromptLengthSJF. The two regimes also separate the gate's

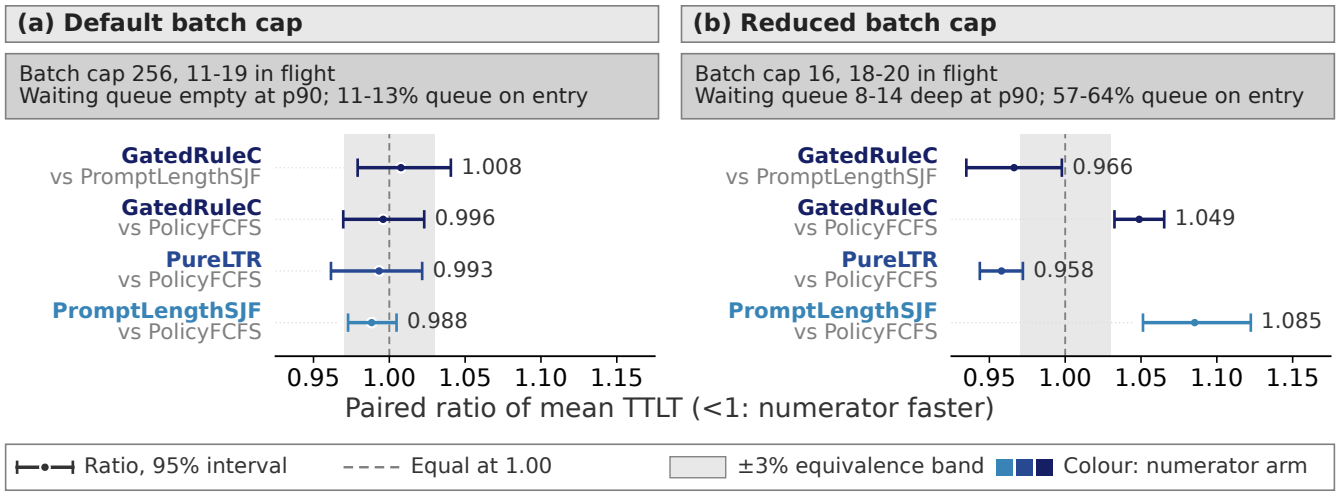


Fig. 7. One engine setting decides the paper's answer: the same four policies on the same workload, with the default running-batch cap no pair separates, and with the cap reduced every pair does. Both panels replay the same 2,025 sessions and each arm resamples within one engine launch (3 replays in (a), 1 in (b)), so neither panel's intervals carry machine-level replication.

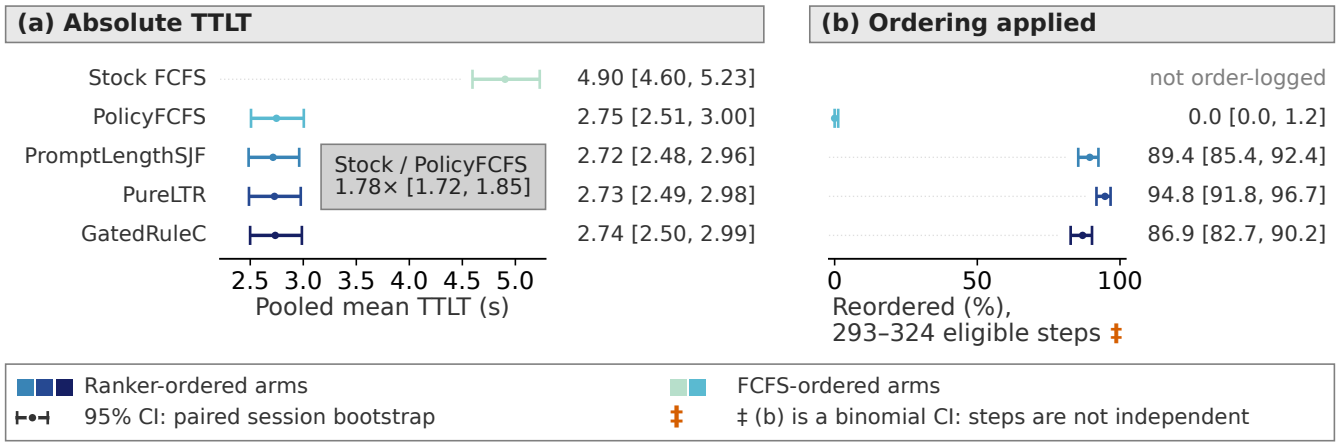


Fig. 8. Serving-level scheduling at the default batch cap: the 1.78× gap between stock and *PolicyFCFS* comes from the scheduler implementation, not from ordering, and the policies rewrote 87–95% of the steps that could express an order. The paired ordering ratios for this regime, and the 3% margin they are judged against, are in Fig. 7(a).

cost: GatedRuleC against PureLTR was 1.003 ($[0.991, 1.014]$) with an empty queue and is 1.095 ($[1.079, 1.112]$) here, and against PolicyFCFS it is 1.049 ($[1.033, 1.065]$), outside the 3% margin. Abstention pins S1, S2 and zero-tool requests in their arrival slots, and once the queue is deep those pinned requests block the ones behind them. Abstaining is free when nothing is waiting and costly when something is.

VI. Discussion

Two configured quantities decide how much of the offline schema signal survives: the decision timeout, which forfeits 34.6% of decisions at the documented 15 ms and none at 75 ms, and the running-batch cap, which decides whether a queue forms at all. Uncapped, ordering changed nothing; capped at 16, the learned order beats arrival order by 4.2% while the free length heuristic runs 8.5% worse than doing nothing.

A. Which Gateway Optimizations Apply, and Which We Did Not Test

Three caches are in play. *Semantic response caching*, the gateway's main lever, was unavailable: its cache excludes streaming requests, and every captured request streamed. *Prefix KV caching* is a different mechanism that E1's byte-identical schema prefix makes plausible: with caching on and nothing else changed, pooled mean TTLT falls by 17% on the two stock-scheduler arms and 22–23% on the four policy-base arms, at a 62% peak hit rate. That run cached the whole

stream, so it did not isolate the schema's share. E3 reports the third, schema-embedding caching, insufficient.

A six-stage run enabled the machinery the ordering rounds disabled: the gateway's scheduler path, queue limits and scorer discipline, at 5.76 requests/s. Mean TTLT never left the run's drift band (largest step 70 ms, 1.6% of the mean), nothing was shed, and goodput was flat. *Queue-triggered scoring* consults the scorer only when the gateway's admission queue forms; that queue did not form at this rate, and the decision services recorded zero scoring requests, so the run measures trigger-path non-activation, not a free scorer.

B. What the Negative Results Buy

Identity features fail because tool combinations essentially never repeat in training data; removing truncation produced no detected improvement; caching fails because a $1.9\times$ speedup does not bridge a $48\times$ budget violation. Neither rescue fits this BERT-base Ranker to the 15 ms default; fitting it needs the GPU placement with a recalibrated budget (75 ms covers every decision) or a path off the critical path, which E6 makes more valuable still.

C. The Baseline Was the Result

The one large effect in our uncontended serving measurements did not come from our contribution. Swapping the scheduler implementation while holding arrival order fixed cut mean time-to-last-token by 44%; changing the order moved it by less than the session drifted over the same hours. An earlier round, benchmarked against the stock scheduler alone, reported uniform gains across every policy we tried. That should have raised suspicion: policies that order differently should not improve identically. Those gains came from the implementation underneath. The same mistake awaits anyone who takes a serving engine's default scheduler as a baseline; the control must share your implementation and differ only in the variable claimed.

D. Limitations

Offline claims rest on one workload family (ToolACE) and one label model, the live traces come from a single agent product, and the 75 captured payloads calibrate the synthetic workload, so they cannot double as a validation set. The replay inherits the capture's compressed job-size variance, which may leave ordering little room to separate.

One choice bounded the first three ordering rounds: vLLM's running-batch cap sat at its 256-slot default, mean occupancy under a thirteenth of it (Sec. II). Mean occupancy does not forbid queueing, and our logs show it did not: bursts put 11.9–12.5% of requests behind another, but those queues drained within a scheduling step instead of accumulating. Our calibration targeted completion (a 95% success criterion), not contention. E6 removes that bound and the ordering result flips, so every ordering number here is conditional on its regime. We did not vary how contention arises: E6 constrains the batch, while contexts large enough to exhaust the KV cache would reach it another way.

The main matrix rests on three replay repetitions inside a single engine launch per arm, and E6 on one cold launch per arm. The Ranker predicts decode length only, so prefill cost is unmodeled; the offline split is row-level; the replay is open-loop; S2 is underpowered at n=78; and the gate's confidence is a measured τ floor, not a calibrated probability. Serving hardware is a vendor-modified 48 GB RTX 4090.

E. Future Work

Six directions would occupy the next six months, in the order we would take them. First, the two attribution ablations still owed: re-parent the shim onto the asynchronous base to confirm the 44%, and an oracle-order arm sorted by measured length to separate residual ranking error from the mechanism E6 exposes. Second, test whether the gate's cost under contention can be recovered by letting abstained requests keep a bounded slot rather than a fixed one. Third, validate queue-triggered scoring under a load that forms the gateway queue. Fourth, close the predictor-cost gap by distillation and grow the S2 stratum to statistical power. Fifth, move to closed-loop session benchmarks. Sixth, explore continual fine-tuning on production traffic under explicit leakage discipline: training and evaluation traffic must never mix.

VII. Conclusion

Agent traffic carries a signal the serving layer currently discards: the tool-schema text attached to every tool-bearing request. Read as text, the request improves output-length ranking by $+0.19$ Kendall's τ over the tuned scalar family we benchmark, and the schema itself contributes $+0.044$ of that; quality changes little on the two adequately powered unseen-tool strata (equivalence untested), where identity features have nothing to look up. Under the gateway's documented 15 ms decision timeout the predictor silently loses 34.6% of its decisions; the two CPU rescues we pre-registered do not repair that, while a 75 ms timeout eliminated logged fail-open in these runs. Whether acting on the signal pays depends on contention, which across our runs was set by configuration rather than by the traffic. With the cap at its default the queue stayed empty at the 90th percentile and no policy separated from another; capped at sixteen, most requests queued, and the learned ranker beat arrival order by 4.2% while prompt-length ordering ran 8.5% slower than leaving the queue alone. The reliability gate follows the same regime split: free when nothing waits and, once something does, costly enough — abstained requests hold their slots and block — to leave the gated system 4.9% slower under contention than the arrival order it was built to improve on. Two levers paid in the uncapped configuration, neither from prediction, and both ship with the engine: its asynchronous scheduler — which our policy base inherits — cut mean time-to-last-token by 44% with arrival order held fixed, and prefix caching over the resent schemas cut a further 17–23%. The schema signal is real, its cheap scalar substitute is worse than doing nothing, and an operator who wants either should first check what the batch cap leaves a scheduler to decide.

Appendix A: Artifact Availability

https://github.com/TaliesinYang/vllm-ltr-optimization

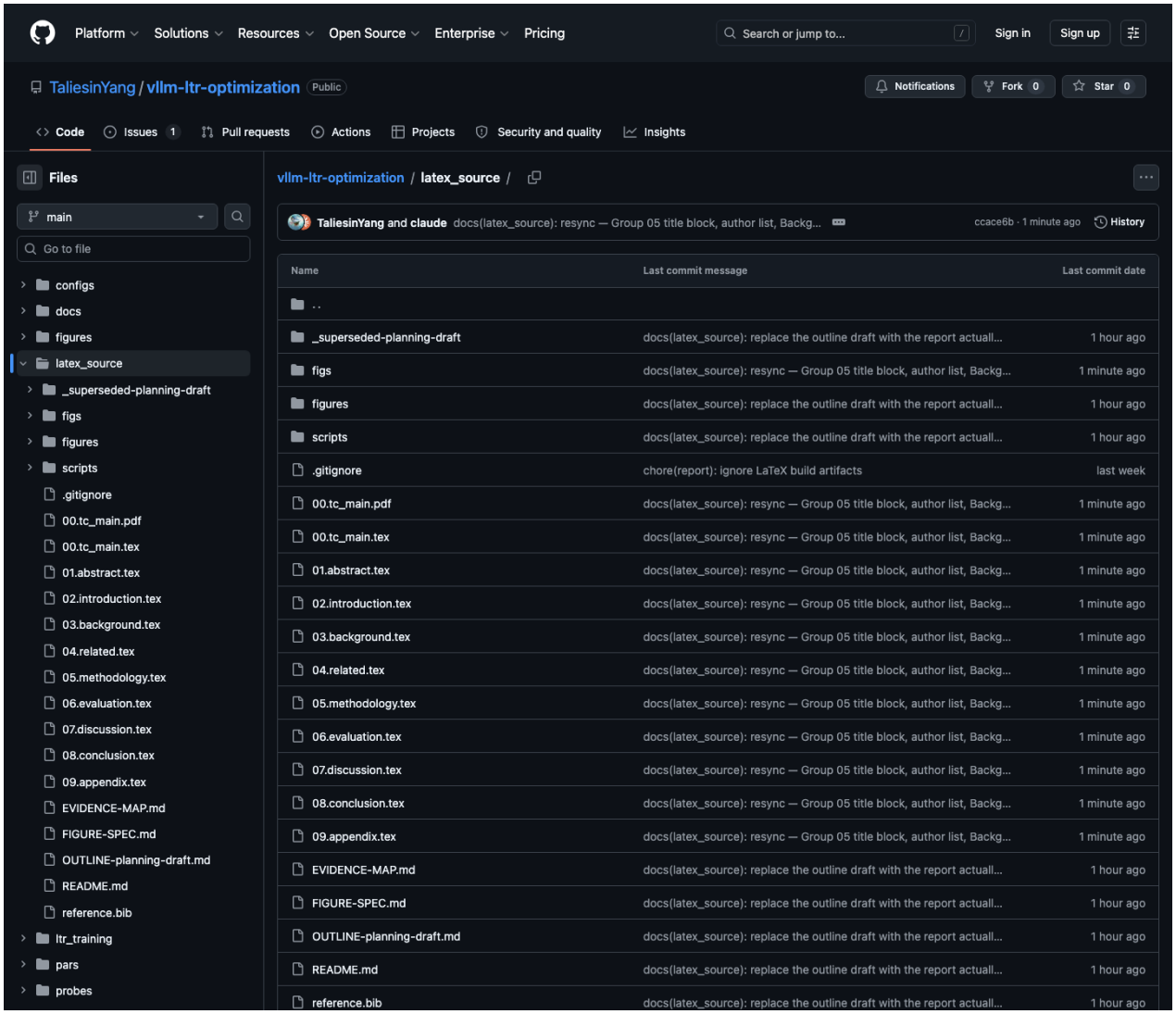


Fig. 9. The `latex_source/` directory of the project repository, holding the LaTeX source, bibliography, figures and build checks that produce this report.

REFERENCES

[1] Y. Fu, S. Zhu, R. Su, A. Qiao, I. Stoica, and H. Zhang, "Efficient llm scheduling by learning to rank," in *NeurIPS 2024*, 2024. [Online]. Available: https://openreview.net/pdf?id=wlLjY10Gi6

[2] Y. Tao, Y. Zhang, M. Dearing, X. Wang, Y. Fan, M. E. Papka *et al.*, "Ranking before serving: Low-latency LLM serving via pairwise learning-to-rank," in *ISC High Performance 2026 Research Paper Proceedings (41st International Conference)*, Hamburg, Germany, Jun. 2026, pp. 1–13.

[3] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A distributed serving system for transformer-based generative models," in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 2022, pp. 521–538.

[4] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu *et al.*, "Efficient memory management for large language model serving with pagedattention," in *Proceedings of the 29th Symposium on Operating Systems Principles*. ACM, 2023.

[5] G. Park, S. Lee, and Y. Park, "Minimizing response latency in LLM-based agent systems: A comprehensive survey," *IEEE Access*, vol. 14, pp. 26 140–26 168, 2026.

[6] H. Qiu, W. Mao, A. Patke, S. Cui, S. Jha, C. Wang, H. Franke, Z. T. Kalbarczyk, T. Başar, and R. K. Iyer, "Efficient interactive llm serving with proxy model-based sequence length prediction," 2024, aSPLOS 2024 AIOps workshop.

[7] M. G. Kendall, "A new measure of rank correlation," *Biometrika*, vol. 30, no. 1/2, pp. 81–93, 1938.

[8] S. Choi, J. Goo, E. Jeon, M. Yang, and M. Jang, "ELIS: Efficient LLM iterative scheduling system with response length predictor," 2025, arXiv preprint. [Online]. Available: https://arxiv.org/abs/2505.09142

[9] Y. Li, Y. Chen, J. Chen, E. Choukse, H. Qiu, G. E. Suh *et al.*, "Beyond prediction: Tail-aware scheduling for llm inference," in *ICML 2026, PMLR 306*, 2026.

[10] B. Wu, Y. Zhong, Z. Zhang, S. Liu, F. Liu, Y. Sun *et al.*, "Fastserve: Iteration-level preemptive scheduling for large language model inference," in *NSDI 2026*, 2026. [Online]. Available: https://www.usenix.org/conference/nsdi26/presentation/wu-bingyang

[11] H. Zheng, Y. Zhang, F. Fu, X. Zhou, H. Luo, H. Zhu *et al.*, "Scheduling llm inference with uncertainty-aware output length predictions," 2026, arXiv preprint. [Online]. Available: https://arxiv.org/abs/2604.00499

[12] W. Zhang, Z. Wu, Y. Mu, R. Ning, B. Liu, N. Sarda *et al.*, "Jitserve: Slo-aware llm serving with imprecise request information," in *NSDI 2026*, 2026. [Online]. Available: https://www.usenix.org/conference/nsdi26/presentation/zhang-wei

[13] M. Luo, X. Shi, C. Cai, T. Zhang, J. Wong, Y. Wang *et al.*, "Agentix: An efficient serving engine for llm agents as general programs," in *NSDI 2026*, 2026. [Online]. Available: https://www.usenix.org/conference/nsdi26/presentation/luo

[14] R. Abhyankar, Z. He, V. Srivatsa, H. Zhang, and Y. Zhang, "Infercept: Efficient intercept support for augmented llm inference," in *ICML 2024, PMLR 235*, 2024.

[15] W. Liu, X. Huang, X. Zeng, X. Hao, S. Yu, D. Li *et al.*, "Toolace: Winning the points of llm function calling," 2024.

[16] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu *et al.*, "ToolLLM: Facilitating large language models to master 16000+ real-world APIs," in *Proceedings of the 12th International Conference on Learning Representations (ICLR)*, 2024.

[17] S. G. Patil, T. Zhang, X. Wang, and J. E. Gonzalez, "Gorilla: Large language model connected with massive APIs," in *Advances in Neural Information Processing Systems 37 (NeurIPS)*, 2024.

[18] S. Agarwal, P. Namyar, A. Wolman, R. Ambavat, A. Gupta, and Q. Zhang, "Switchcraft: AI model router for agentic tool calling," 2026.

[19] S. G. Patil, H. Mao, F. Yan, C. C.-J. Ji, V. Suresh, I. Stoica *et al.*, "The Berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models," in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, ser. Proceedings of Machine Learning Research, vol. 267. PMLR, 2025, pp. 48 371–48 392.

[20] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang *et al.*, "Sglang: Efficient execution of structured language model programs," in *Advances in Neural Information Processing Systems 37 (NeurIPS)*, 2024.

[21] H. Zhou, M. Chen, Q. Lin, Y. Wang, X. She, S. Liu *et al.*, "Overload control for scaling wechat microservices," in *Proceedings of the ACM Symposium on Cloud Computing (SoCC)*, 2018, pp. 149–161.

[22] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, 2019, pp. 4171–4186.

[23] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma *et al.*, "Lightgbm: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems 30*, 2017.

[24] B. Efron, "Bootstrap methods: Another look at the jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.